

Iterative Programming in R

Learning Objectives

A systematic study of this chapter will help the reader understand the following concepts:

- While and for loops
- Loops for vectors, matrices, data frames, and lists

5.1 INTRODUCTION

In most of the problems that we come across, iterative statements are necessary. These include for and while loops. In this chapter, we present how to program in R using these loops.

5.2 WHILE LOOP

The syntax for the while loop consists of *test_expression* that acts as the minimal condition for the loop to check how many times it has to be executed.

```
while(test_expression){
    statement
}
```

Example 5.1 Write an R program to print the values in vectors using the while loop.

Solution:

In Fig. 5.1, a vector is first initialized with two values, "Hello" and "while loop". These vector values are printed inside the while loop for five times, with an initial counter *cnt* set to 2.

```
Console ~/ >
> v <- c("Hello","while loop")
> cnt <- 2
>
> while (cnt < 7) {
+   print(v)
+   cnt = cnt + 1
+ }
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
[1] "Hello"      "while loop"
```

Fig. 5.1 Hello world using while loop

5.3 FOR LOOP

The syntax of for loop consists of *variable* that can be used inside the loop for iterating over the list of elements in a vector or a list. It also consists of *start* and *end* that represents the

```
for (variable in start:end){
    statement
}
```

Example 5.2 Write an R program to print the values in vectors using the for loop.

Solution:

In Fig. 5.2, a vector is first initialized with two values "Hello" and "for loop". These vector values are printed inside the for loop for six times, with an initial counter *cnt* set to 2.

```
Console ~/
> v <- c("Hello", "for loop")
> for (i in 2:7){print(v)}
[1] "Hello"      "for loop"
[1] "Hello"      "for loop"
[1] "Hello"      "for loop"
[1] "Hello"      "for loop"
[1] "Hello"      "for loop"
[1] "Hello"      "for loop"
```

Fig. 5.2 Hello world using for loop

5.4 LOOPING OVER LIST

In R programming, lists may be in the form of vectors, arrays, or data frames. In the following sub-sections, we demonstrate the usage of loops with these data structures in R.

5.4.1 Loops for Vectors

A vector can be created in R using the *c()* function with any kind of values in it. In Fig. 5.3, a vector *v* is created with four values 1, 2, One, and True. A for loop or a while loop can be used to print the values of the vector as shown in Fig. 5.4. The difference between the for and while loops is that the control variable in the while loop needs to be initialized and controlled appropriately.

```
Console ~/
> v <- c(1, 2, "One", "True")
> for (i in v){print(i)}
[1] "1"
[1] "2"
[1] "One"
[1] "True"
```

Fig. 5.3 Vectors using for loop

```

Console ~/
> v <- c(1,2,"One","True")
> cnt<-1
> while(cnt<length(v))
+ {
+   print(v)
+   cnt<-cnt+1
+ }
[1] "1"      "2"      "One"    "True"
[1] "1"      "2"      "One"    "True"
[1] "1"      "2"      "One"    "True"

```

Fig. 5.4 Printing vectors using while loop

5.4.2 Loops for Matrices

A matrix can be created in R using the *matrix()* function. In Fig. 5.5, a matrix *y* is created with 5 rows and 4 columns. The square brackets *[]* are used for accessing the elements of the matrix. A for loop can be used for printing the elements of the matrix in the following way. Here, two loops need to be used—one for iterating over the rows and another for iterating over the columns.

```

Console ~/
> y<-matrix(1:20,nrow=5,ncol=4)
> for(i in 1:nrow(y))
+ {
+   for(j in 1:ncol(y))
+   {
+     print(y[i,j])
+   }
+ }
[1] 1
[1] 6
[1] 11
[1] 16
[1] 2
[1] 7
[1] 12
[1] 17
[1] 3
[1] 8
[1] 13
[1] 18
[1] 4
[1] 9
[1] 14
[1] 19
[1] 5
[1] 10
[1] 15
[1] 20

```

Fig. 5.5 Printing the elements of a matrix using for loop

```

for (i in 1:nrow(y)){
  for(j in 1:ncol(y)){
    print(y[i,j])
  }
}

```

5.4.3 Loops for Data Frames

A data frame in R consists of a group of vectors. It is created using the `data.frame()` function. The argument passed to the data frame needs to be a list of vectors separated by commas.

Example 5.3 Write an R program to create four vectors namely *patientid*, *age*, *diabetes*, and *status*. Put these four vectors into a data frame *patientdata* and print the values using a for loop.

Solution: As shown in Fig. 5.6, four vectors named *patientid*, *age*, *diabetes*, and *status* are created initially. These four vectors are put into a data frame named *patientdata*. We can use the for loop for iterating over the items of the data frame as shown. In order to iterate over the columns of the *patientdata* frame, the `names()` function is used.

```

Console ~/
> patientID <- c(1,2,3,4)
> age <- c(25,34,28,52)
> diabetes <- c("Type1","Type2","Type1","Type1")
> status<- c("Poor","Improved","Excellent","Poor")
> patientdata<-data.frame(patientID,age,diabetes,status)
> for (i in names(patientdata)){
+ print(patientdata[i])}
patientID
1      1
2      2
3      3
4      4
age
1 25
2 34
3 28
4 52
diabetes
1 Type1
2 Type2
3 Type1
4 Type1
status
1 Poor
2 Improved
3 Excellent
4 Poor

```

Fig. 5.6 Creation of `data.frame()`

In this example, the contents of the data frame are listed one by one according to the list of vectors. One more alternative way of displaying the contents of the data frame using the for loop is the use of `{[ ]}` (two square brackets) for accessing the elements of the data frame. The difference is that, in this method, the name of the columns in the data frame are not displayed, whereas in Example 5.3, they are displayed.

```

Console ~/
> for ( i in names(patientdata)){
+ print(patientdata[[i]])}
[1] 1 2 3 4
[1] 25 34 28 52
[1] Type1 Type2 Type1 Type1
Levels: Type1 Type2
[1] Poor      Improved  Excellent
[4] Poor
3 Levels: Excellent ... Poor

```

Fig. 5.7 Listing the contents of *data.frame()*

5.4.4 Loops for Lists

A list is created in R using the *list()* function. In Fig. 5.8, two vectors *one* and *two* are created and passed as arguments to the list *new*. In order to iterate over the elements of the list, the *names()* function is used to identify the vectors in the list. The for loop is used to iterate over the elements of the list one by one and \$ indicates the vectors passed to the list.

```

Console ~/
> new<-list(one=c("1","2","three"))
> for ( i in names(new)){
+ print(new[i])}
$one
[1] "1"      "2"      "three"

$two
[1] "one"    "two"    "three"

```

Fig. 5.8 Creation of *list()*

Example 5.4 Write an R program to print HELLO 10 times using the for loop.

Solution:

Figure 5.9 shows how to print "Hello" 10 times in R using a for loop. The for loop is initialized with the starting condition as 1 and the terminating condition as 10.

```

Console ~/
> for ( i in 1:10){
+   print("Hello World!")
+ }
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"
[1] "Hello World!"

```

Fig. 5.9 Printing HELLO 10 times

Example 5.5 Write an R program to print if the given number is ODD/EVEN.

Solution:

Figure 5.10 displays whether a given number is odd/even using a function. A vector *x* is initialized with 10 values and the function *is.even()* checks whether the given number is even and *is.odd()* function checks whether the given number is odd. It will list either *FALSE* or *TRUE* depending on the validity of the condition.

```

Console ~/
> x <- 1:10
> is.even <- function(x) x %% 2 == 0
> is.even(x)
[1] FALSE TRUE FALSE TRUE FALSE
[6] TRUE FALSE TRUE FALSE TRUE
> is.odd <- function(x) x %% 2 != 0
> is.odd(x)
[1] TRUE FALSE TRUE FALSE TRUE
[6] FALSE TRUE FALSE TRUE FALSE

```

Fig. 5.10 Printing if given number is odd/even

Example 5.6 Write an R program to print the Fibonacci series.

Solution:

Figure 5.11 shows the code to print the first 10 Fibonacci numbers in R. A numeric vector *fibvals* is initialized with a length of 10. A for loop is iterated from 3 to length with the expression *fibvals[i]=fibvals[i-1]+fibvals[i-2]*, that is, a Fibonacci number is calculated as the sum of the previous two Fibonacci numbers.

```

Console ~/
> len <- 10
> fibvals <- numeric(len)
> fibvals[1] <- -1
> len <- 10
> fibvals <- numeric(len)
> fibvals[1] <- 0
> fibvals[2] <- 1
> for(i in 3:len){
+ fibvals[i] <- fibvals[i-1]+fibvals[i-2]}
> fibvals
[1] 0 1 1 2 3 5 8 13 21 34

```

Fig. 5.11 Printing Fibonacci series

Example 5.7 Write an R program to print the first 100 prime numbers.

Solution:

Figure 5.12 prints the first 100 prime numbers. A function *is.prime()* checks whether a number in the vector is prime or not. A vector *n* of 100 numbers is passed as an argument to the function *is.prime()*. The *vectorize()* function converts the returned result of the function *is.prime()* to a vector.

```

Console ~/
> n <- 1:100
> is.prime1 <- function(num){
+   if(num==2){
+     TRUE
+   }else if(any(num %% 2:(num-1) == 0))
+     FALSE
+   }else{
+     TRUE
+   }
+ }
> is.prime1 <- Vectorize(is.prime1)
> n[is.prime1(n)]
[1]  2  3  5  7 11 13 17 19 23 29 31
[12] 37 41 43 47 53 59 61 67 71 73 79
[23] 83 89 97

```

Fig. 5.12 Printing the first 100 prime numbers

Example 5.8 Write an R program to print the sum of the first 100 numbers.

Solution:

Figure 5.13 displays the sum of the first 100 numbers using a for loop. A vector *n* of length 100 is initialized first and a loop is used to iterate over these numbers to calculate the sum of the numbers in the vector.

```

Console ~/
> n <- 1:100
> sum=0
> sum_of <- function(num){
+   for (num in n){
+     sum=sum+num
+   }
+   print(sum)
+ }
> sum_of(n)
[1] 5050

```

Fig. 5.13 Printing the sum of the first 100 numbers

Example 5.9 Write an R program to calculate the factorial of a number.

Solution:

Figure 5.14 displays the factorial of a number. A function *fact_num()* calculates the factorial of a number using a for loop. The loop starts with a condition from 1 until *num* with the expression *factorial = factorial * i* that calculates the factorial of a number.

Console -/ ↻

```

> fact_num<-function(num){
+   factorial=1
+   if(num < 0){
+     print("Sorry, factorial does not exist for negative numbers")
+   } else if (num == 0){
+     print("The factorial of 0 is 1")
+   }
+   else{
+     for (i in 1:num){
+       factorial = factorial * i
+     }
+     print(paste("The factorial of ",num, "is",factorial))
+   }
+ }
> fact_num(3)
[1] "The factorial of 3 is 6"

```

Fig. 5.14 Calculating the factorial of a number

Example 5.10 Write an R program to calculate the LCM of two numbers.

Solution:

Figure 5.15 displays the LCM of two numbers using a while loop. The function *lcm()* calculates the LCM of two numbers where the while() loop is iterated until it breaks at the condition of TRUE.

Console -/ ↻

```

> lcm <- function(x,y){
+   if ( x >y) {
+     greater =x
+   } else {
+     greater =y
+   }
+   while(TRUE) {
+     if((greater %% x ==0) && (greater %% y == 0)) {
+       lcm =greater
+       break
+     }
+     greater = greater +1
+   }
+   return (lcm)
+ }
> lcm(3,2)
[1] 6

```

Fig. 5.15 LCM calculation

Example 5.11 Write an R program to calculate the GCD of two numbers.

Solution:

Figure 5.16 displays the GCD of two numbers using the `ifelse()` function. Here, *test_expression* must be a logical vector (or an object that can be coerced to logical). The return value is a vector with the same length as *test_expression*. This returned vector has the element from *x* if the corresponding value of *test_expression* is TRUE or from *y* if the corresponding value of *test_expression* is FALSE.

```

Console ~/
> gcd <- function(x,y){
+   r <- x %%y
+   return (ifelse(r,gcd(y,r),y))
+ }
> gcd(4,6)
[1] 2

```

Fig. 5.16 GCD calculation

Programming Questions

- 5.1 Write a program using for loop that displays the elements in the vector
`y <- c("q", "w", "e", "r", "z", "c")`.
- 5.2 Write a nested loop, where the outer for() loop increments "a" 3 times, and the inner for() loop increments "b" 3 times. The break statement exits the inner for() loop after 2 increments. The nested loop prints the values of variables, a and b.
- 5.3 Using the following variable `x = c(2, 4)`, type a while() loop that adds even numbers to x, while the length of x is less than 12. For example, in the first iteration you get `x = 2, 4, 6` and in the third iteration `x = 2, 4, 6, 8`.
- 5.4 Write a program to initialize a vector of centimetres and convert it into meters using a for loop.
- 5.5 Write a program to initialize a data frame for 5 people with columns as name, gender, and height (cm). Use for loop, to convert the height into meters and add an extra column to the existing data frame as (height:meters).
- 5.6 Write a program to initialize a vector (5, 9, 81, 100, 10^6). Use a for loop to calculate the square root of each element in the vector and put it a new vector.

Concept Assessment Questions

- 5.1 Write a program in R to check if the given input number from the user is a prime number or not.
- 5.2 Write a program in R to find the sum of the first 100 natural numbers.
- 5.3 Write a program to find the sum of odd and even numbers from the first 50 natural numbers.
- 5.4 Write an R program using loops to print the cube of each number from 1 to 10.
- 5.5 What is the output of the following R code?

```

A <- c("Delhi", "Bangalore",
"Chennai", "Mumbai", "Kolkata",
"Hyderabad")
for(j in 1:6) {
    print(A[j])
}

```
- 5.6 What is the output of the following R code?

```

b = rep(0,10)
for (i in 1:10)

```